

Análise e Desenvolvimento de Sistemas

Disciplina: Sistemas Operacionais II

Parte 2

Fatec Lins

Professor: Alexandre Ponce de Oliveira

Sumário

1. SISTEMA DE ARQUIVOS	4
1.1 O comando fdisk	4
1.2 O comando du	6
1.3 O comando df	6
1.4 O comando mount e umount	7
1.4 Gerenciador de pacotes	8
1.5 O comando dpkg	10
2. GERENCIAMENTO DE USUÁRIOS	12
2.1 O comando useradd	12
2.2 O comando passwd	13
2.3 O comando userdel	14
2.4 O comando usermod	14
2.5 O comando id	15
2.6 O comando groupadd	15
2.7 O comando groupdel	16
2.8 O comando groupmod	16
2.9 O comando groups	17
3. GERENCIAMENTO DE PROCESSOS	18
3.1 Comando ps	18
3.2 O comando kill	20
3.3 O comando killall	21
3.4 Comando top	21
3.5 O comando jobs	22
3.6 O comando bg	23
3.7 O comando fg	23
3.8 O comando nice	24
3.9 O comando renice	25
4. PROCESSO DE INICIALIZAÇÃO	26
4.1 O processo init	26
4.2 Scripts de inicialização	27
4.3 Agendamento de scripts com crontab	29
4.4 Outros scripts	31
5. INTERFACE DE REDE	32
5.1 O comando ifconfig	32
5.2 O comando ping	34
6. BIBLIOGRAFIA	36

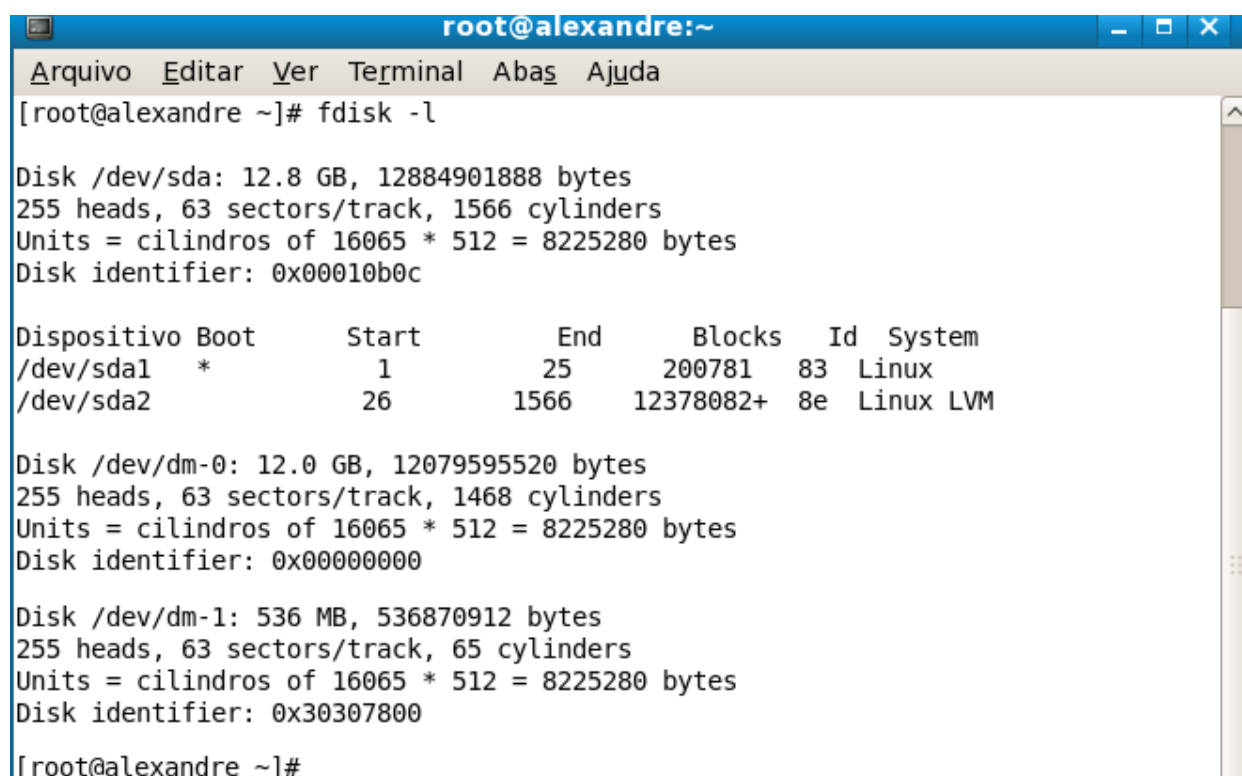
1. Sistema de Arquivos

Um dos principais aspectos para uma administração coerente do Linux é o entendimento de como este lida com os dispositivos de armazenamento e as ferramentas utilizadas para garantir a segurança e a integridade do sistema. A seguir veremos os comandos relacionados ao Sistema de Arquivos.

1.1 O comando fdisk

O comando **fdisk** (Fixed Disk – Disco Fixo) é uma ferramenta para manipulação das partições do Linux. A sintaxe é:

```
fdisk [opções] [dispositivo]
```



```
root@alexandre:~# fdisk -l

Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cilindros of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00010b0c

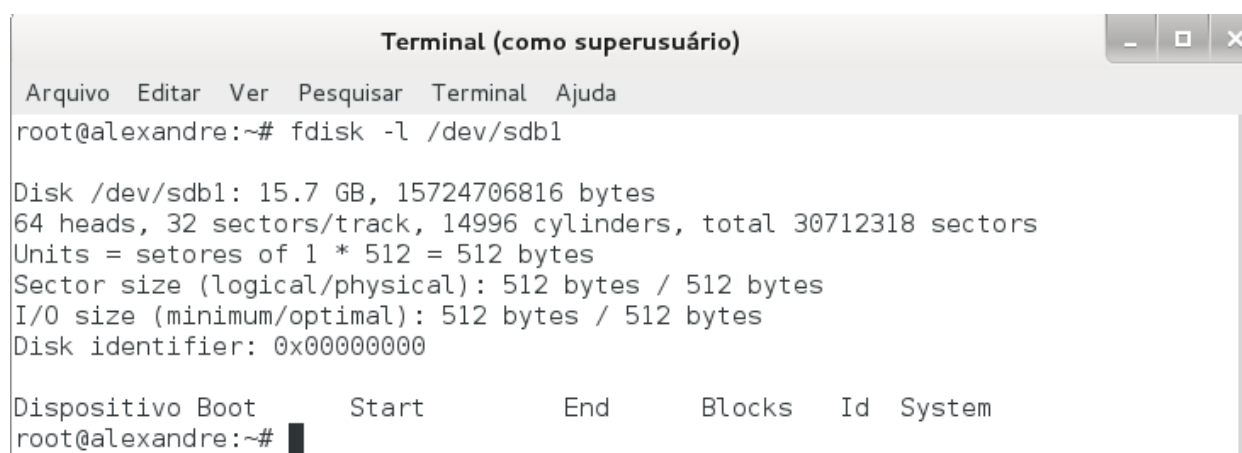
Dispositivo Boot      Start          End      Blocks    Id System
/dev/sda1  *           1           25       200781    83  Linux
/dev/sda2                26        1566     12378082+   8e  Linux LVM

Disk /dev/dm-0: 12.0 GB, 12079595520 bytes
255 heads, 63 sectors/track, 1468 cylinders
Units = cilindros of 16065 * 512 = 8225280 bytes
Disk identifier: 0x00000000

Disk /dev/dm-1: 536 MB, 536870912 bytes
255 heads, 63 sectors/track, 65 cylinders
Units = cilindros of 16065 * 512 = 8225280 bytes
Disk identifier: 0x30307800

[root@alexandre ~]#
```

A figura acima lista as partições correntes de um Sistema Linux, pode-se notar duas partições sendo a primeira a partição principal e a segunda a partição de boot. A figura abaixo lista a partição de um pendrive que foi conectado pela porta USB.



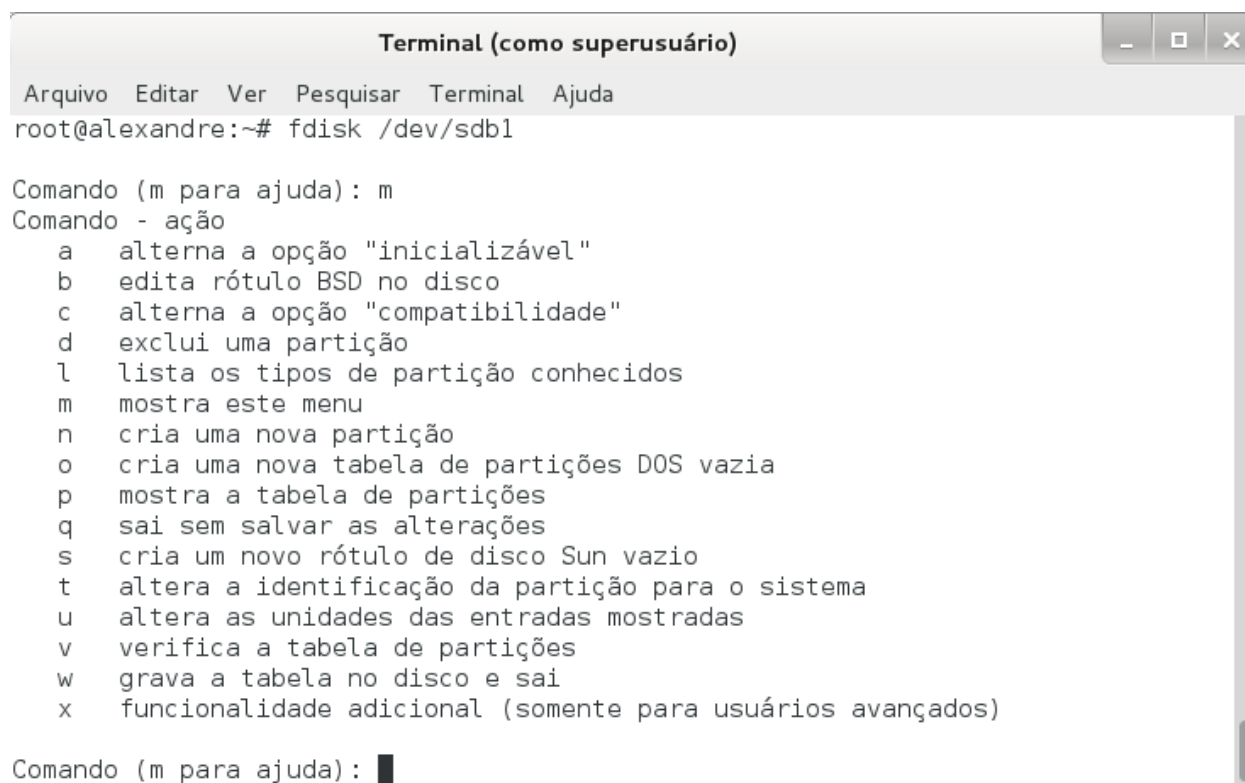
```
Terminal (como superusuário)

root@alexandre:~# fdisk -l /dev/sdb1

Disk /dev/sdb1: 15.7 GB, 15724706816 bytes
64 heads, 32 sectors/track, 14996 cylinders, total 30712318 sectors
Units = setores of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

Dispositivo Boot      Start          End      Blocks    Id System
root@alexandre:~#
```

A figura abaixo mostra as opções disponíveis para manipular as partições de um disco, por exemplo, n – cria uma partição; d – exclui uma partição; p – mostra a tabela de partições.



```

Terminal (como superusuário)
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~# fdisk /dev/sdb1

Comando (m para ajuda): m
Comando - ação
  a  alterna a opção "inicializável"
  b  edita rótulo BSD no disco
  c  alterna a opção "compatibilidade"
  d  exclui uma partição
  l  lista os tipos de partição conhecidos
  m  mostra este menu
  n  cria uma nova partição
  o  cria uma nova tabela de partições DOS vazia
  p  mostra a tabela de partições
  q  sai sem salvar as alterações
  s  cria um novo rótulo de disco Sun vazio
  t  altera a identificação da partição para o sistema
  u  altera as unidades das entradas mostradas
  v  verifica a tabela de partições
  w  grava a tabela no disco e sai
  x  funcionalidade adicional (somente para usuários avançados)

Comando (m para ajuda): █

```

A seguir é mostrado a sequencia de comandos para criar uma partição no pendrive (/dev/sdb1):

```

Comando (m para ajuda): n
Partition type:
  p  primary (0 primary, 0 extended, 4 free)
  e  extended
Select (default p): p
Número da partição (1-4, default 1): 1
Primeiro setor (2048-30712317, default 2048):
Using default value 2048
Last setor, +setores or +size{K,M,G} (2048-30712317, default 30712317):
Using default value 30712317

```

A figura abaixo mostra a partição criada:

```

Comando (m para ajuda): p

Disk /dev/sdb1: 15.7 GB, 15724706816 bytes
64 heads, 32 sectors/track, 14996 cylinders, total 30712318 sectors
Units = setores of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

Dispositivo Boot      Start      End      Blocks  Id  System
/dev/sdb1p1          2048    30712317    15355135  83  Linux

```

Por fim, a opção para excluir uma partição:

```
Comando (m para ajuda): d
Selected partition 1

Comando (m para ajuda): p

Disk /dev/sdb1: 15.7 GB, 15724706816 bytes
64 heads, 32 sectors/track, 14996 cylinders, total 30712318 sectors
Units = setores of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000

Dispositivo Boot      Start          End      Blocks    Id  System
```

1.2 O comando du

O comando **du** (Disk Usage – Disco Usado) mostra a utilização de espaço em disco para arquivos e diretórios.

du [opções]

Segue abaixo as opções do comando **du**:

Opção	Descrição
-h	Mostra em formato mais legível (por exemplo, 1K 86Mb, 5Gb)
-s	Mostra apenas o sumário da pesquisa
-b	Exibe tamanhos em bytes, em vez de kbytes
-m	Exibe tamanhos em megabytes
-a	Mostra os espaços utilizados pelos arquivos e diretórios

Exemplo utilizando o comando **du -ha**:

```
root@alexandre:/etc/httpd/conf
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre conf]# du -ha
20K      ./magic
4,0K     ./certificados/chave_privada.key
4,0K     ./certificados/chave_publica.key
4,0K     ./certificados/req_certificado
4,0K     ./certificados/servidor.crt
4,0K     ./certificados/chave_privada.key.backup
24K      ./certificados
40K      ./httpd.conf
92K      .
[root@alexandre conf]#
```

1.3 O comando df

O comando **df** (Disk Free – Disco Livre) mostra as partições, inclusive o uso atual e espaço livre, ou seja, a capacidade do HD.

df [opções] [arquivo]

Sem utilizar nenhuma opção o comando **df** irá mostrar os espaços usados e livres de cada sistema de arquivos montados naquele momento. Os resultados serão mostrados normalmente em

blocos de 1024 bytes por padrão. Se for informado o arquivo, somente as informações pertinentes a este sistema de arquivos serão exibidas. As opções são as seguintes:

Opção	Descrição
-h	Mostra em formato mais legível (por exemplo, 1K 86Mb, 5Gb)
-k	Exibe tamanhos em kbytes
-a	Lista todos os sistemas de arquivos, mesmo com tamanho igual a zero

Abaixo uma figura com o comando **df**:

```

root@alexandre:/etc/httpd/conf
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre conf]# df
Sist. Arq.          1K-blocos      Usad Dispon.   Uso% Montado em
/dev/mapper/VolGroup00-LogVol00
                    11427040    5928112   4909104   55% /
/dev/sda1           194442      12379    172024    7% /boot
tmpfs               127628        12     127616    1% /dev/shm
[root@alexandre conf]#

```

Outro exemplo, usando a opção **-h**:

```

root@alexandre:/etc/httpd/conf
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre conf]# df -h
Sist. Arq.          Tam    Usad Disp   Uso% Montado em
/dev/mapper/VolGroup00-LogVol00
                    11G   5,7G   4,7G   55% /
/dev/sda1           190M    13M   168M    7% /boot
tmpfs               125M    12K   125M    1% /dev/shm
[root@alexandre conf]#

```

1.4 O comando mount e umount

O comando **mount** serve para tornar disponível um sistema de arquivos dentro de outro sistema de arquivos.

mount [-t tipo] [device] [ponto de montagem]

Exemplo: montagem do pendrive (sdb1) no diretório /mnt/pendrive

```

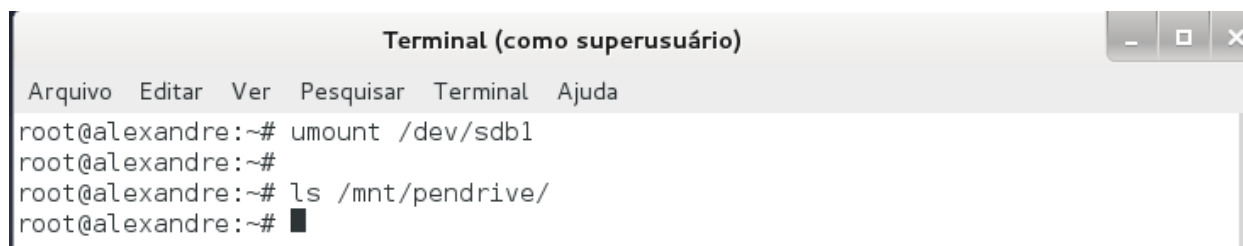
Terminal (como superusuário)
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~# mkdir /mnt/pendrive
root@alexandre:~#
root@alexandre:~# mount -t vfat /dev/sdb1 /mnt/pendrive/
root@alexandre:~#
root@alexandre:~# ls -l /mnt/pendrive/
total 88
drwx----- 2 alexandre alexandre 8192 Mar  9 15:15 Arquivos
drwx----- 2 alexandre alexandre 8192 Nov 21 16:59 Codigos
drwx----- 2 alexandre alexandre 8192 Feb 26 09:59 Teste
drwx----- 4 alexandre alexandre 8192 Nov 21 15:27 urDrive
root@alexandre:~#

```

O comando **umount** é utilizado para desmontar um sistema de arquivos previamente montado.

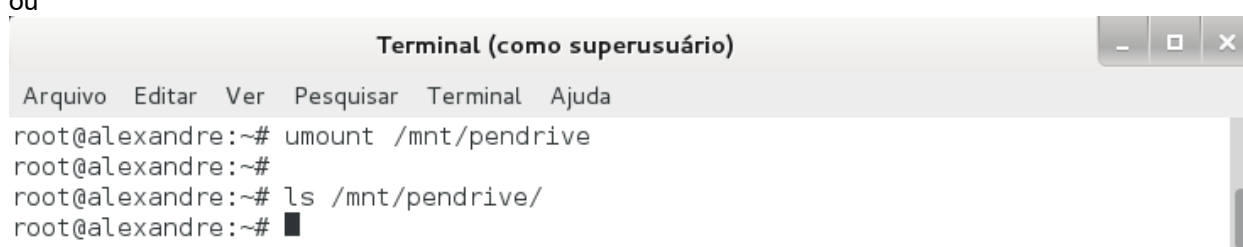
```
umount [device] ou
umount [ponto de montagem]
```

Exemplo: desmontagem do pendrive (sdb1) no diretório /mnt/pendrive



```
Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# umount /dev/sdb1
root@alexandre:~#
root@alexandre:~# ls /mnt/pendrive/
root@alexandre:~# █
```

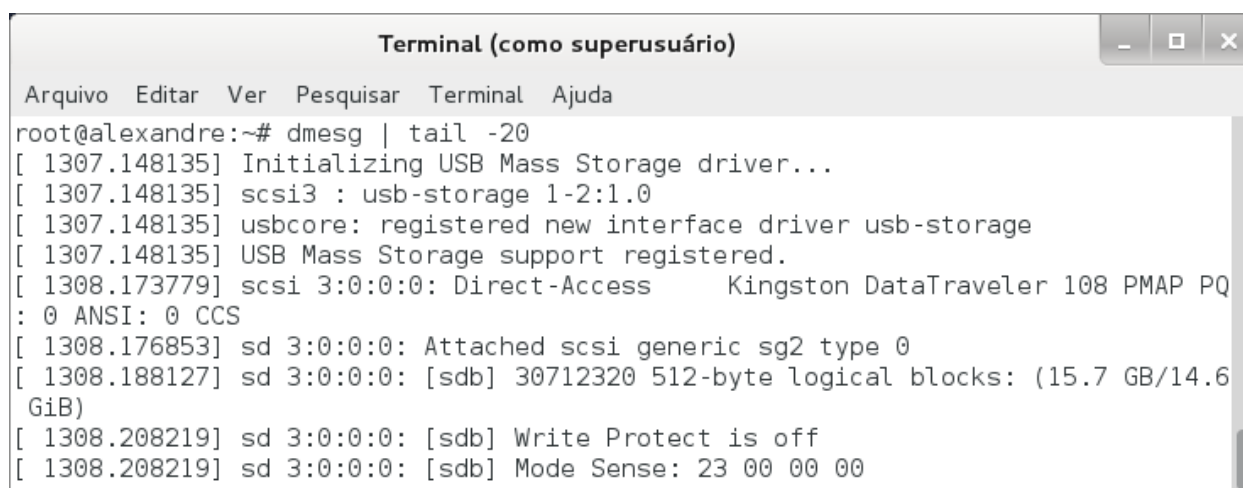
ou



```
Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# umount /mnt/pendrive
root@alexandre:~#
root@alexandre:~# ls /mnt/pendrive/
root@alexandre:~# █
```

Embora continuem sendo importantes e bastante úteis para solucionar problemas, tais comandos são cada vez menos usados devido aos sistemas de montagem automática utilizados nas distribuições.

Alternativamente, o comando **dmesg** pode identificar qual dispositivo foi associado ao pendrive. A seguir um exemplo do comando:



```
Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# dmesg | tail -20
[ 1307.148135] Initializing USB Mass Storage driver...
[ 1307.148135] scsi3 : usb-storage 1-2:1.0
[ 1307.148135] usbcore: registered new interface driver usb-storage
[ 1307.148135] USB Mass Storage support registered.
[ 1308.173779] scsi 3:0:0:0: Direct-Access Kingston DataTraveler 108 PMAP PQ
: 0 ANSI: 0 CCS
[ 1308.176853] sd 3:0:0:0: Attached scsi generic sg2 type 0
[ 1308.188127] sd 3:0:0:0: [sdb] 30712320 512-byte logical blocks: (15.7 GB/14.6
GiB)
[ 1308.208219] sd 3:0:0:0: [sdb] Write Protect is off
[ 1308.208219] sd 3:0:0:0: [sdb] Mode Sense: 23 00 00 00
```

1.4 Gerenciador de pacotes

O sistema de empacotamento usa um banco de dados para saber quais pacotes estão instalados, quais não estão e quais estão disponíveis para instalação. O **apt-get** usa esse banco de dados para saber instalar os pacotes solicitados pelo usuário e para saber quais pacotes são necessários para que o pacote selecionado funcione corretamente. Para atualizar essa lista, utilize o comando:

```
# apt-get update
```

Este comando procura pelas listas de pacotes nos repositórios no arquivo `/etc/apt/sources.list` e verifica possíveis atualizações nos pacotes, principalmente de segurança. A seguir é mostrado o conteúdo do arquivo **sources.list**.

```
alexandre@alexandre: /etc/bind
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~#
root@alexandre:~# cd /etc/apt
root@alexandre:/etc/apt#
root@alexandre:/etc/apt# head -9 sources.list
deb cdrom:[Debian GNU/Linux 7.6.0 _Wheezy_ - Official i386 DVD Binary-1 20
140712-13:02]/ wheezy contrib main

deb cdrom:[Debian GNU/Linux 7.6.0 _Wheezy_ - Official i386 DVD Binary-2 20
140712-13:02]/ wheezy contrib main

deb cdrom:[Debian GNU/Linux 7.6.0 _Wheezy_ - Official i386 DVD Binary-3 20
140712-13:02]/ wheezy contrib main

deb http://security.debian.org/ wheezy/updates main contrib
deb-src http://security.debian.org/ wheezy/updates main contrib

root@alexandre:/etc/apt#
```

Para fazer a instalação de um pacote, com o arquivo **sources.list** preparado e a lista de pacotes disponíveis, execute o seguinte comando:

```
alexandre@alexandre: /etc/bind
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~# apt-get install squid
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os pacotes extra a seguir serão instalados:
  squid-common squid-langpack
Pacotes sugeridos:
  squidclient squid-cgi logcheck-database resolvconf
Os NOVOS pacotes a seguir serão instalados:
  squid squid-common squid-langpack
0 pacotes atualizados, 3 pacotes novos instalados, 0 a serem removidos e 2
06 não atualizados.
É preciso baixar 1.400 kB de arquivos.
Depois desta operação, 8.177 kB adicionais de espaço em disco serão usados
.
Você quer continuar [S/n]?
```

O APT vai buscar no banco de dados a versão mais nova do pacote e vai baixá-la do repositório correspondente no arquivo **sources.list**. Se esse pacote depender de outro(s) pacote(s), as dependências são detectadas e instaladas junto. Na instalação do pacote **squid**, outros pacotes dependentes foram instalados (**squid-common** e **squid-langpack**).

Para remover um pacote instalado sem dependência pode ser utilizado a opção **remove**. Se outros pacotes foram instalados devido a dependências é indicado a opção **autoremove**. A seguir é mostrado o processo de desinstalação com as duas opções:


```
alexandre@alexandre: /etc/bind
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# apt-get remove mc
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
0 seguinte pacote foi instalado automaticamente e já não é necessário:
  mc-data
Utilize 'apt-get autoremove' para o remover.
Os pacotes a seguir serão REMOVIDOS:
  mc
0 pacotes atualizados, 0 pacotes novos instalados, 1 a serem removidos e 0
não atualizados.
Depois desta operação, 1.349 kB de espaço em disco serão liberados.
Você quer continuar [S/n]? █
```

```
alexandre@alexandre: /etc/bind
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# apt-get autoremove mc
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os pacotes a seguir serão REMOVIDOS:
  mc mc-data
0 pacotes atualizados, 0 pacotes novos instalados, 2 a serem removidos e 0
não atualizados.
Depois desta operação, 6.269 kB de espaço em disco serão liberados.
Você quer continuar [S/n]? █
```

1.5 O comando dpkg

O comando **dpkg** é usado para gerenciar pacotes (arquivos) Debian no sistema. Os pacotes (arquivos) **.deb**, são por exemplo, instalados, removidos com o **dpkg**. O comando é limitado pois não tem informações de pacotes disponíveis, e ocorre falha se uma dependência não for satisfeita.

Para fazer a instalação de um pacote Debian (arquivo .deb), execute o seguinte comando:

```
# dpkg -i pacote.deb
```

Para remover um pacote Debian (arquivo .deb), execute o seguinte comando:

```
# dpkg -r pacote
```

```
alexandre@alexandre: /etc/bind
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# dpkg -r mc
(Lendo banco de dados ... 150561 ficheiros e directórios actualmente instala
dos.)
Removendo mc ...
update-alternatives: a usar /usr/bin/vim.tiny para disponibilizar /usr/bin
/view (view) em modo automático
Processando gatilhos para menu ...
Processando gatilhos para man-db ...
root@alexandre:~# █
```

A opção **-r** não remove completamente o pacote, pois todos os arquivos de configuração e arquivos de log permanecem. Para uma desinstalação completa utilize a opção **-P**.

```
alexandre@alexandre: /etc/bind
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~# dpkg -P mc
(Lendo banco de dados ... 150561 ficheiros e directórios actualmente instalados.)
Removendo mc ...
update-alternatives: a usar /usr/bin/vim.tiny para disponibilizar /usr/bin/view (view) em modo automático
Expurgando arquivos de configuração de mc ...
Processando gatilhos para menu ...
Processando gatilhos para man-db ...
root@alexandre:~#
```

Para listar os pacotes instalados use a opção **-l**.

```
alexandre@alexandre: /etc/bind
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
root@alexandre:~# dpkg -l | grep "mc"
ii  libclutter-imcontext-0.1-0:i386  0.1.4-3
    i386      Open GL based interactive canvas library IMContext framework
ii  libclutter-imcontext-0.1-bin  0.1.4-3
    i386      Open GL based interactive canvas library IMContext framework
ii  libxdmcp6:i386  1:1.1.1-1
    i386      X11 Display Manager Control Protocol library
ii  libxvmc1  2:1.0.7-1+deb7u2
    i386      X11 Video extension library
ii  pcmciautils  018-8
    i386      PCMCIA utilities for Linux 2.6
ii  spamc  3.3.2-5+deb7u1
    i386      Client for SpamAssassin spam filtering daemon
root@alexandre:~#
```

2. Gerenciamento de Usuários

O linux é um sistema operacional multiusuário, por isso todos os usuários devem ser cadastrados e tenham permissões de acesso diferenciadas. É possível também cadastrá-los em grupos para facilitar o gerenciamento.

2.1 O comando useradd

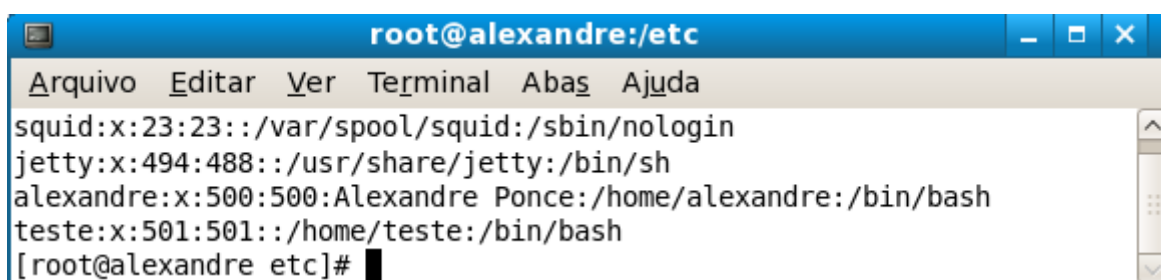
O comando **useradd** permite criar usuários. Somente o superusuário poderá utilizar este comando. A sintaxe é:

```
useradd [opções] [usuário]
```

A execução deste comando impacta na alteração dos seguintes arquivos:

- /etc/passwd – informações de contas de usuários;
- /etc/shadow – senhas criptografadas;
- /etc/group – informações de grupos.

O comando cria uma entrada para o usuário no arquivo /etc/passwd, contendo o nome do usuário, o diretório padrão, entre outras informações. Na figura abaixo temos um exemplo do final do arquivo passwd:



```

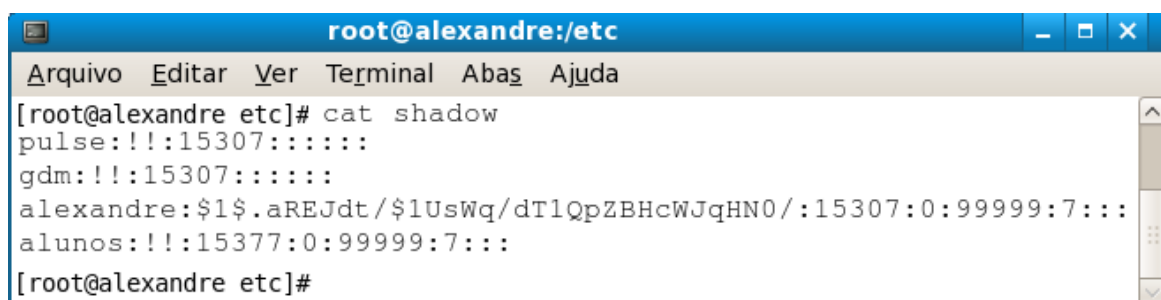
root@alexandre:/etc
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
squid:x:23:23::/var/spool/squid:/sbin/nologin
jetty:x:494:488::/usr/share/jetty:/bin/sh
alexandre:x:500:500:Alexandre Ponce:/home/alexandre:/bin/bash
teste:x:501:501::/home/teste:/bin/bash
[root@alexandre etc]#

```

As informações são separadas por colunas pelo caractere ":", a seguir tem-se o significado de cada uma das colunas:

- 1ª coluna: login do usuário;
- 2ª coluna: significa que password está armazenado no arquivo /etc/shadow;
- 3ª coluna: UID (User IDentification), ou seja, o número de identificação do usuário;
- 4ª coluna: GID (Group IDentification), ou seja, o número de identificação do grupo do usuário;
- 5ª coluna: Informações complementares do usuário, por exemplo, o nome completo;
- 6ª coluna: o diretório padrão do usuário; e
- 7ª coluna: shell do usuário, ou seja, o interpretador de comandos que o usuário utilizará.

Na figura abaixo temos um exemplo do final do arquivo shadow, as informações também são separadas por colunas pelo caractere ":". Logo após a figura tem-se o significado de cada uma das colunas:



```

root@alexandre:/etc
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre etc]# cat shadow
pulse:!!:15307:::::::
gdm:!!:15307:::::::
alexandre:$1$.aREJdt/$1UsWq/dTlQpZBHcWJqHN0/:15307:0:99999:7:::
alunos:!!:15377:0:99999:7:::
[root@alexandre etc]#

```

- 1ª coluna: login do usuário;

2ª coluna: senha criptografada, caso o valor seja “!” ou “*” a conta está desabilitada, caso o valor seja “!!” a senha ainda não foi definida;
 3ª coluna: data da última alteração da senha, considerando o número de dias a partir de 1º de janeiro de 1970;
 4ª coluna: número de dias antes que a senha pode ser alterada;
 5ª coluna: número de dias antes que uma alteração de senha seja solicitada;
 6ª coluna: número de dias após a senha expirar antes que a conta seja desabilitada;
 7ª coluna: data, em forma de número de dias, desde que a conta do usuário foi desabilitada; e
 8ª coluna: é um campo reservado, sem utilização.

O comando **useradd** utiliza os seguintes arquivos, porém sem efetuar alterações:

- `/etc/default/useradd` – informações padrão de usuários;
- `/etc/login.defs` – definições de login para o sistema;
- `/etc/skel` – diretório contendo arquivos de padrões(modelos).

Abaixo temos a tabela de opções:

Opção	Descrição
<code>-d dir</code>	Define em “dir” qual será o diretório padrão do usuário.
<code>-g grupo</code>	Grupo ou número do grupo inicial ao qual o usuário pertencerá. Normalmente as distribuições criam o grupo com o mesmo nome do usuário.
<code>-s shell</code>	Informa qual o shell a ser usado no login.
<code>-e</code>	Define a data de expiração da conta, sendo no formato MM/DD/AA.

Exemplos:

```
[root@teste etc]$ useradd teste
```

Cria o usuário **teste**

```
[root@teste etc]$ useradd -d /home/ponce alexandre
```

Cria o usuário **alexandre** e define o seu diretório home como sendo `/home/ponce` e não o default que seria `/home/alexandre`.

```
[root@teste etc]$ useradd -s /bin/false aluno
```

Cria o usuário **aluno** e coloca um shell falso, ou seja, sem shell (comum para usuários de e-mails apenas).

```
[root@teste etc]$ useradd -e 04/30/13 aluno2
```

Cria o usuário **aluno2** e define que a conta vai expirar em 04/30/13, sendo MM/DD/AA.

```
[root@teste etc]$ useradd -g admin joao
```

Cria o usuário **joao** e define como grupo padrão **admin**. O grupo deve ser criado primeiro.

2.2 O comando passwd

O comando **passwd** permite que se troque a senha de determinado usuário. O superusuário pode trocar a senha de qualquer outro. O usuário comum pode trocar somente a sua senha. As senhas são armazenadas no arquivo `/etc/shadow`, e as demais informações dos usuários são armazenadas no arquivo `/etc/passwd`.

Após a criação do usuário será necessário criar uma senha, caso contrário, não será permitido que este usuário faça *login* no sistema. Sua sintaxe é:

```
passwd [usuário]
```

Exemplos:

```
[root@teste etc]$ passwd aluno
```

O superusuário irá alterar a senha do usuário **aluno**.

```
[aluno@teste aluno]$ passwd
```

O usuário **aluno** irá alterar sua própria senha.

```
[aluno@teste aluno]$ passwd -d aluno
```

Apaga a senha, deixando-a em branco.

```
[aluno@teste aluno]$ passwd -x 30 aluno
```

Define a senha do usuário aluno com validade de 30 dias.

Observação importante: O sistema não aceita que sejam criadas senhas muito simples. Recomenda-se sempre misturar letras e números para criar sua senha e, se possível, misturar também letras maiúsculas e minúsculas.

2.3 O comando userdel

O comando **userdel** permite que se elimine usuários do sistema. Somente o superusuário poderá utilizar este comando. A sintaxe é:

```
userdel [opções] [usuário]
```

Exemplos:

```
[root@teste etc]$ userdel aluno
```

Elimina a conta do usuário **aluno**.

```
[root@teste etc]$ userdel -r aluno
```

Elimina a conta do usuário **aluno** e todo o conteúdo do diretório pessoal do respectivo usuário.

2.4 O comando usermod

O comando **usermod** permite alterar dados da conta do usuário, tais como ID, shell, grupo, entre outros. Somente o superusuário poderá utilizar este comando. A sintaxe é:

```
usermod [opções] [usuário]
```

Abaixo temos a tabela de opções:

Opção	Descrição
-d dir [-m]	Cria um novo diretório home para o usuário, a opção -m faz com que o diretório atual do usuário seja movido para o novo diretório.
-g grupo	Altera o grupo padrão do usuário.
-G grupo	Define outro(s) grupo(s) ao(s) qual(is) o usuário pertencerá.
-s shell	Altera o shell do usuário.
-u uid	Altera o UID do usuário.
-l usuário	Altera o nome do usuário.
-e MM/DD/AA	Define a data de expiração da conta.

Exemplos:

```
[root@teste etc]$ usermod -s /bin/false aluno
```

Altera o shell do usuário **aluno** para falso, ou seja, sem shell.

```
[root@teste etc]$ usermod -l ponce alexandre
```

Alterar o nome de login atual (**alexandre**) para o novo (**ponce**), permanecendo a senha.

```
[root@teste etc]$ usermod -G aluno nelson
```

Adiciona o usuário **nelson** ao grupo **aluno**.

2.5 O comando id

Utiliza-se o comando **id** para verificar informações sobre os usuários do sistema. Ele nos fornece dados como a identificação do usuário no sistema (UID) e os grupos que ele pertence (GID). A sintaxe é:

```
id [opções] nome
```

Caso nenhum nome seja informado, o **id** retorna os dados do usuário corrente.

Exemplo:

```
[aluno@teste aluno]$ id
uid=503(aluno) gid=503(aluno) grupos=503(aluno)
```

```
[root@teste etc]$ id root
uid=0(root) gid=0(root) grupos=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk)
```

Abaixo temos a tabela de opções:

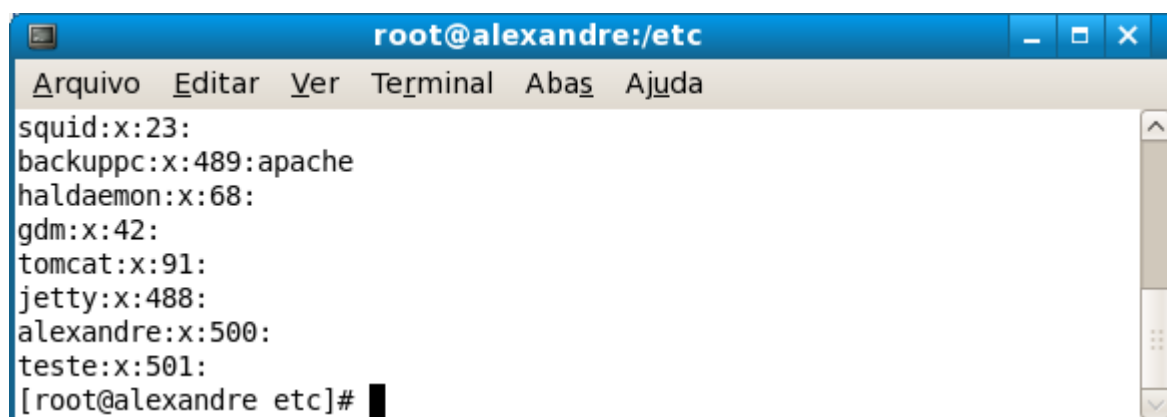
Opção	Descrição
-g / --group	Lista somente o ID do grupo.
-G / --groups	Lista somente os grupos suplementares.
-n / --name	Lista o nome do usuário ou grupo em vez do número ID. Utiliza-se em conjunto com -u, -g ou -G.
-u / --user	Lista somente o ID do usuário.

2.6 O comando groupadd

Para facilitar a administração do sistema, podem-se usar grupos de usuários com perfis semelhantes. Por exemplo, definir grupos conforme os departamentos da empresa. A sintaxe do comando é:

```
groupadd [opções] grupo
```

O arquivo que contém a relação dos grupos é o **/etc/group**. Abaixo um exemplo do arquivo:

A terminal window titled 'root@alexandre:/etc' with a menu bar containing 'Arquivo', 'Editar', 'Ver', 'Terminal', 'Abas', and 'Ajuda'. The terminal displays the contents of the /etc/passwd file, listing system users (squid, backuppc, haldaemon, gdm, tomcat, jetty) and regular users (alexandre, teste). The prompt is '[root@alexandre etc]#'.

```
root@alexandre:/etc
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
squid:x:23:
backuppc:x:489:apache
haldaemon:x:68:
gdm:x:42:
tomcat:x:91:
jetty:x:488:
alexandre:x:500:
teste:x:501:
[root@alexandre etc]#
```

Exemplo:

```
[root@teste etc]$ groupadd alunos
```

Cria o grupo **alunos**.

```
[root@teste etc]$ groupadd -f alunos
```

Cria o grupo **alunos**, mesmo se o grupo já existir.

2.7 O comando groupdel

Este comando exclui um grupo. A sintaxe do comando é:

```
groupdel grupo
```

Exemplo:

```
[root@teste etc]$ groupdel alunos1
```

Exclui o grupo **alunos1**

Observação importante: Não pode remover o grupo primário de um usuário, portanto, remova o usuário primeiro.

2.8 O comando groupmod

Este comando altera um grupo existente. A sintaxe do comando é:

```
groupmod [opções] grupo
```

Exemplos:

```
[root@teste etc]$ groupmod -n novo antigo
```

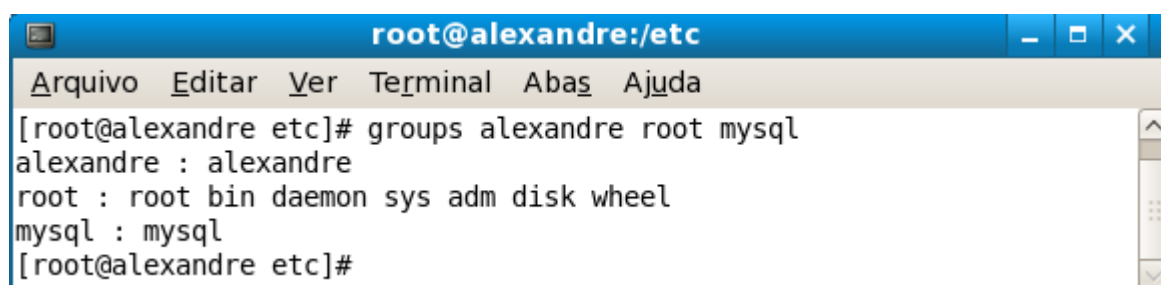
Altera o nome do grupo **antigo** para o **novo**.

```
[root@teste etc]$ groupmod -g 512 professor
```

Altera o ID do grupo **professor** para o novo ID, 512.

2.9 O comando groups

Este comando mostra o(s) grupo(s) ao(s) qual(is) um ou mais usuários pertencem, conforme figura abaixo:



```
root@alexandre:/etc
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre etc]# groups alexandre root mysql
alexandre : alexandre
root : root bin daemon sys adm disk wheel
mysql : mysql
[root@alexandre etc]#
```


3. Gerenciamento de Processos

O linux é um sistema operacional multitarefa, executa diversos processos simultaneamente. Geralmente um processo é um programa em execução, embora possa ser apenas parte de um programa mais complexo.

Para cada processo, o sistema fornece um código de identificação (PID – Process Identifier), existe também uma hierarquia entre os processos, onde um processo pode chamar outro, que por sua vez pode chamar um terceiro, e assim por diante. Pode-se ter processos em primeiro plano (foreground) e também em segundo plano (background). Um exemplo bem fácil do que é o foreground é quando se executa um comando demorado, sendo necessário esperar seu término para se fazer outra coisa, isso no mesmo terminal. Para se executar este mesmo comando em background, é só colocar o **&** no final do comando. Uma outra forma de suspender um processo é teclando a sequência de teclas **CTRL+Z**, por exemplo, está sendo executado o comando **vi**, se dentro do programa for digitada a sequência de teclas o processo será suspenso (colocado em segundo plano). Abaixo um exemplo:

The screenshot shows a terminal window titled 'root@alexandre:~'. The menu bar includes 'Arquivo', 'Editar', 'Ver', 'Terminal', 'Abas', and 'Ajuda'. The user runs the command `find /etc -name ifcfg-eth0`. The terminal shows the command is running, then it is interrupted by `CTRL + Z`, resulting in `[1]+ Stopped find /etc -name ifcfg-eth0`. A blue arrow points from the text 'CTRL + Z (segundo plano)' to the 'Stopped' status. After pressing `CTRL + Z`, the user runs `ps`, which displays the following output:

```

[1]+  Stopped                  find /etc -name ifcfg-eth0
[root@alexandre ~]# ps
  PID TTY          TIME CMD
 2370 pts/0        00:00:00 bash
 2529 pts/0        00:00:00 find
 2530 pts/0        00:00:00 ps
[root@alexandre ~]#

```

Ao usar um editor é fácil controlá-lo, indicar que ele deve carregar arquivos entre outras funções. Isso é possível porque o editor oferece algumas opções para fazê-lo, e por que o editor é diretamente interligado a um terminal. Alguns programas não são projetados de forma a utilizar informações contínuas do usuário, e assim ele se desconecta do terminal em sua primeira oportunidade. Por exemplo, um servidor HTTP passa o tempo todo respondendo requisições da rede, e normalmente ele não necessita intervenção alguma do usuário. Programas que transportam mensagens de correio de um local para outro é outro exemplo deste tipo aplicação. Chamamos estes programas de *daemons*.

Algumas vezes será necessário se comunicar com um processo daemon. Essa comunicação é chamada de sinais, pode-se comunicar com o daemon (ou qualquer outro processo em execução) enviando-o um sinal. Existe uma série de sinais que podem ser enviados aos processos. Você só pode enviar um sinal para os processos que te pertencem. Se você tentar enviar sinais para um processo que não é seu terá permissão negada. A única exceção é o usuário root, que pode enviar sinais para os processos de todos.

3.1 Comando ps

O comando **ps** (Process Status) mostra os processos ativos do sistema. A sintaxe é:

ps [opções]

Abaixo temos a tabela de opções:

Opção	Descrição
-a	Mostra os processos de todos os usuários.
-f	Mostra os processos em formato de "árvore", ou seja, com seus caminhos completos.
-u	Inclui na lista os usuários e hora do início do processo.
-x	Inclui processos não associados a um terminal.

Exemplo:

```
[root@teste etc]$ ps
```

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ps
  PID TTY          TIME CMD
 2370 pts/0    00:00:00 bash
 4260 pts/0    00:00:00 ps
[root@alexandre ~]#

```

Exibe os processos ativos do usuário.

```
[root@teste etc]$ ps -aux
```

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ps -aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
ND
root         1   0.0  0.2   2112    592 ?        Ss   17:05   0:01 init
root         2   0.0  0.0      0      0 ?        S<   17:05   0:00 [kth]
root         3   0.0  0.0      0      0 ?        S<   17:05   0:00 [mig]
root         4   0.0  0.0      0      0 ?        S<   17:05   0:00 [kso]
root         5   0.0  0.0      0      0 ?        S<   17:05   0:00 [wat]
root         6   0.0  0.0      0      0 ?        S<   17:05   0:00 [eve]
root         7   0.0  0.0      0      0 ?        S<   17:05   0:00 [khe]
root        58   0.0  0.0      0      0 ?        S<   17:05   0:00 [kbl]
root        61   0.0  0.0      0      0 ?        S<   17:05   0:00 [kac]
apache    2690   0.0  2.0  28012   5288 ?        S    18:40   0:00 /usr/
root     6196   0.0  0.4   4652   1092 ?        SN   18:49   0:00 /bin/
root     6197   0.0  0.4   4652   1060 ?        SN   18:49   0:00 /bin/
root     6198   0.0  0.2   4320    708 ?        SN   18:49   0:00 awk -
root     6203   0.0  0.1   3796    464 ?        SN   18:49   0:00 sleep
root     6216   1.0  0.3   4528    936 pts/0    R+   18:53   0:00 ps -a
[root@alexandre ~]#

```

Exibe todos os processos, inclusive os processos não associados a terminais específicos, mostrando seus respectivos usuários.

Os principais campos dessa listagem são:

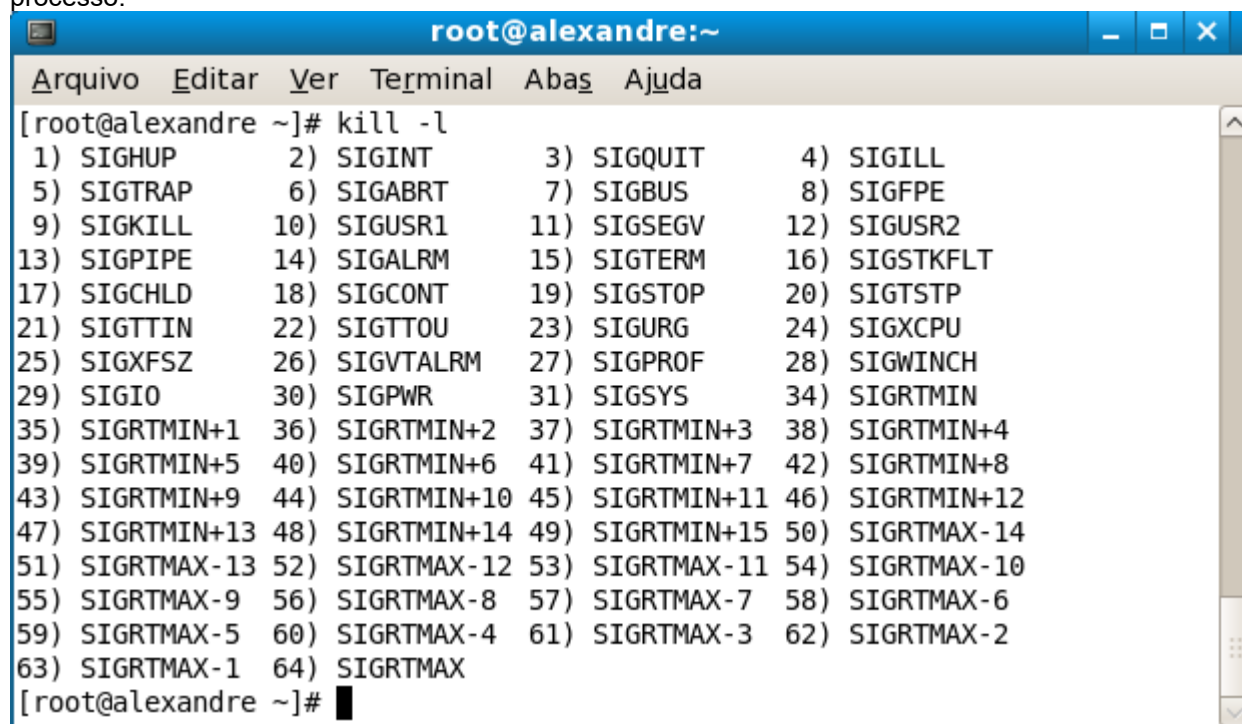
- USER: proprietário do processo.
- PID: número do processo.
- %CPU: porcentagem da CPU usada pelo processo.
- %MEM: porcentagem da memória usada pelo processo.
- VSZ: memória total usada pelo processo.
- RSS: memória física (RAM) usada pelo processo.
- TTY: terminal ao qual o processo está ligado.
- STAT: status do processo.
- START: data de lançamento do processo.
- TIME: tempo total de CPU usado pelo processo.
- COMMAND: comando usado para lançar o processo.

3.2 O comando kill

O comando **kill** permite que o superusuário ou os donos dos processos possam eliminar processos ativos. A sintaxe do comando é:

```
kill [opções] [-sinal] [processo]
```

Utilizando a opção **-l**, podemos visualizar todos os sinais que podem ser enviados para um processo.



```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL
 5) SIGTRAP     6) SIGABRT     7) SIGBUS      8) SIGFPE
 9) SIGKILL    10) SIGUSR1    11) SIGSEGV    12) SIGUSR2
13) SIGPIPE    14) SIGALRM    15) SIGTERM    16) SIGSTKFLT
17) SIGCHLD    18) SIGCONT    19) SIGSTOP    20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU
25) SIGXFSZ    26) SIGVTALRM  27) SIGPROF    28) SIGWINCH
29) SIGIO      30) SIGPWR     31) SIGSYS     34) SIGRTMIN
35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3 38) SIGRTMIN+4
39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12
47) SIGRTMIN+13 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14
51) SIGRTMAX-13 52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10
55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7  58) SIGRTMAX-6
59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
[root@alexandre ~]#

```

Exibe todos os sinais possíveis para enviar ao processo.

Todos os sinais passam pelo kernel, quando enviado qualquer sinal para um processo, este sinal vai para o kernel que o envia para o processo destino.

O sinal **SIGKILL**, quando enviado para um processo, termina a sua execução. Na realidade o que acontece é que o kernel, quando recebe o sinal, não o repassa para o processo, mas sim, remove o processo diretamente.

O sinal **SIGSTOP** suspende a execução de um programa. Quando existe um programa em execução em foreground e deseja-se suspender o seu processamento basta enviar esse sinal.

Caso o comando seja executado sem o parâmetro **-sinal**, será enviado ao processo o sinal **SIGTERM** (terminate).

Outro sinal muito usado é o **SIGHUP**, cujo objetivo é fazer com que o processo leia novamente os seus arquivos de configuração, como se estivesse sendo inicializados. Isso permite que se altere a configuração do sistema ou de serviços do sistema sem que eles tenham que ser desativados.

Exemplo:

```
[root@teste etc]$ kill -9 23857
```

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ps
  PID TTY          TIME CMD
 2370 pts/0    00:00:00 bash
 6253 pts/0    00:00:00 find
 6255 pts/0    00:00:00 ps
[root@alexandre ~]# kill -9 6253
[root@alexandre ~]# ps
  PID TTY          TIME CMD
 2370 pts/0    00:00:00 bash
 6256 pts/0    00:00:00 ps
[1]+  Morto                  find /etc -name ifcfg-eth0
[root@alexandre ~]#

```

Termina a execução do processo especificado (find).

3.3 O comando killall

Apaga todos os processos em andamento com um nome em comum, sem a necessidade de saber o **PID**. Caso exista mais de um processo com o mesmo nome, os dois serão finalizados.

Exemplo:

```
[root@teste ~]$ killall httpd
```

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ps aux | grep httpd
root      3445  6.0  4.1 27112 10564 ?        Ss   19:32   0:00 /usr/sbin/httpd
apache    3447  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3448  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3449  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3450  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3451  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3452  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3453  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
apache    3454  0.0  2.0 27112  5324 ?        S    19:32   0:00 /usr/sbin/httpd
root      3457  0.0  0.2  4072   736 pts/0    S+   19:32   0:00 grep httpd
[root@alexandre ~]# killall httpd
[root@alexandre ~]# ps aux | grep httpd
root      3462  0.0  0.2  4068   732 pts/0    S+   19:33   0:00 grep httpd
[root@alexandre ~]# killall httpd

```

Finaliza todos os processos do serviço **httpd**.

3.4 Comando top

O comando **top** mostra uma lista (atualizada constantemente) dos processos ativos no sistema. A diferença é que a lista é ordenada pelo consumo de recursos da CPU, conforme figura abaixo:

```

root@alexandre:~
Arquivo Editar Ver Terminal Abas Ajuda
top - 19:09:26 up 2:03, 2 users, load average: 0.59, 0.44, 0.53
Tasks: 123 total, 4 running, 119 sleeping, 0 stopped, 0 zombie
Cpu(s): 4.2%us, 3.8%sy, 0.0%ni, 92.0%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 255260k total, 166468k used, 88792k free, 552k buffers
Swap: 524280k total, 70216k used, 454064k free, 58216k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM   TIME+ COMMAND
 2119 root        20   0 37176 8780 4848 R   5.0   3.4   0:32.19 X
 2309 root        20   0 17332 5128 4176 S   1.0   2.0   0:01.94 gnome-screensa
 2234 root        20   0 109m  39m  10m S   0.7  15.9   0:14.83 nautilus
 2365 root        20   0 84028 16m  9568 R   0.7   6.8   0:05.83 gnome-terminal
 6266 root        20   0 2272 1008  784 R   0.3   0.4   0:00.21 top
    1 root        20   0 2112   592  524 S   0.0   0.2   0:01.01 init
    2 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 kthreadd
    3 root        RT  -5     0     0     0 S   0.0   0.0   0:00.00 migration/0
    4 root        15  -5     0     0     0 S   0.0   0.0   0:00.00 ksoftirqd/0
    5 root        RT  -5     0     0     0 S   0.0   0.0   0:00.00 watchdog/0
    6 root        15  -5     0     0     0 S   0.0   0.0   0:00.23 events/0
    7 root        15  -5     0     0     0 S   0.0   0.0   0:00.01 khelper
   58 root        15  -5     0     0     0 S   0.0   0.0   0:00.23 kblockd/0

```

A sintaxe é: `top [opções]`

Abaixo temos a tabela de opções:

Opção	Descrição
-d n	Atualiza a lista a cada n segundos.
-i	Não inclui na lista processos ociosos.

```
[root@teste etc]$ top -i
```

```

root@alexandre:~
Arquivo Editar Ver Terminal Abas Ajuda
top - 19:10:23 up 2:04, 2 users, load average: 0.39, 0.41, 0.51
Tasks: 123 total, 1 running, 122 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.0%us, 0.3%sy, 0.0%ni, 99.7%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 255260k total, 166184k used, 89076k free, 572k buffers
Swap: 524280k total, 70216k used, 454064k free, 58320k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM   TIME+ COMMAND
 6272 root        20   0 2272 1008  784 R   0.3   0.4   0:00.06 top

```

Mostra apenas os processos que estão utilizando CPU ou Memória, ou seja, que estão sendo usados no momento.

3.5 O comando jobs

O comando **jobs** exibe os processos parados ou em execução que estão em segundo plano.

Sintaxe: `jobs [opções]`

Abaixo temos a tabela de opções:

Opção	Descrição
-l	Exibe também os identificadores dos processos (PIDs) dos jobs em segundo plano.

Exemplo:

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# vi teste.txt
[1]+  Stopped                  vi teste.txt
[root@alexandre ~]# top&
[2] 3484
[root@alexandre ~]# ls -l / -R > ls.txt &
[3] 3487

[2]+  Stopped                  top
[root@alexandre ~]# jobs -l
[1]-  3483 Parado              vi teste.txt
[2]+  3484 Parado (saída tty)   top
[3]   3487 Running             ls --color=tty -l / -R > ls.txt &
[root@alexandre ~]#
  
```

3.6 O comando bg

O comando **bg** é usado para reativar um processo e o colocar em segundo plano, na qual, anteriormente foi parado digitando **CTRL+Z**.

Sintaxe: **bg % [JobID]**

Exemplo:

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ls -l / -R > ls.txt &
[2] 3541
[root@alexandre ~]# jobs -l
[1]+  3483 Parado              vi teste.txt
[2]-  3541 Running             ls --color=tty -l / -R > ls.txt &
[root@alexandre ~]# kill -19 3541
[root@alexandre ~]# jobs -l
[1]-  3483 Parado              vi teste.txt
[2]+  3541 Parado (sinal)      ls --color=tty -l / -R > ls.txt
[root@alexandre ~]# bg %2
[2]+  ls --color=tty -l / -R > ls.txt &
[root@alexandre ~]# jobs -l
[1]+  3483 Parado              vi teste.txt
[2]-  3541 Running             ls --color=tty -l / -R > ls.txt &
[root@alexandre ~]# █
  
```

3.7 O comando fg

Ao contrário do comando **bg**, o comando **fg** é usado para colocar o job em primeiro plano.

Sintaxe: **fg % [JobID]**

Exemplo:

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# jobs -l
[2]+  6289 Parado                  find / -name ifcfg-eth0
[root@alexandre ~]# fg %2
find / -name ifcfg-eth0
/etc/sysconfig/network-scripts/ifcfg-eth0
/etc/sysconfig/networking/profiles/default/ifcfg-eth0
/etc/sysconfig/networking/devices/ifcfg-eth0
[root@alexandre ~]#

```

3.8 O comando nice

Todo processo quando criado possui uma determinada prioridade. Esta prioridade pode ser ajustada através do comando **nice**, sendo que este ajuste pode variar entre -20 (alta prioridade) e 19 (baixa prioridade).

Sintaxe: **nice [opções] [comando/programa]**

Exemplo:

```

root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
top - 19:01:12 up 14 min,  6 users,  load average: 2.33, 3.18, 1.92
Tasks: 123 total,  4 running, 119 sleeping,  0 stopped,  0 zombie
Cpu(s): 65.2%us, 33.3%sy,  1.4%ni,  0.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:   255260k total,  250428k used,    4832k free,    844k buffers
Swap:  524280k total,  76988k used,   447292k free,   43768k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 2355 root        20   0 89820  46m  15m  R  19.3  18.6   0:59.41 tomboy
 2139 root        20   0 38764  10m  4272  S   3.3   4.1   0:26.22 X
 2541 root        20   0  2272  1004   780  R   0.7   0.4   0:00.07 top
   431 root        15  -5     0     0     0  S   0.3   0.0   0:01.05 scsi_eh_2
 1950 root        20   0  3388   856   728  S   0.3   0.3   0:01.08 hald-addon-stor
 2319 root        39  19 92452  65m 5088  R   0.3  26.4   0:01.09 yum-updatesd-he
    1 root        20   0  2112   560   512  S   0.0   0.2   0:00.75 init
[root@alexandre ~]# nice -n +10 top
  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 2355 root        20   0 94216  48m  15m  R  21.9  19.5   1:09.94 tomboy
 2139 root        20   0 38912  10m  4304  S   6.6   4.2   0:28.86 X
 2301 root        20   0 44844  10m  9388  S   0.7   4.4   0:00.35 wnck-applet
 2369 root        20   0 84980  18m  10m  S   0.7   7.5   0:00.91 gnome-terminal
 2541 root        20   0  2272  1008   780  R   0.3   0.4   0:00.12 top
 2549 root        30  10  2272  1004   780  S   0.3   0.4   0:00.01 top
    1 root        20   0  2112   552   512  S   0.0   0.2   0:00.75 init
 127 root        15  -5     0     0     0  S   0.0   0.0   0:00.00 cqueue/0

```

3.9 O comando renice

O comando `renice` altera a prioridade de processos que já estão sendo executados. As prioridades variam da mesma forma que o comando `nice`.

Sintaxe: **renice [opções] [processo]**

Exemplo:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2355	root	20	0	89820	46m	15m	R	19.3	18.6	0:59.41	tomboy
2139	root	20	0	38764	10m	4272	S	3.3	4.1	0:26.22	X
2571	root	20	0	2272	1004	780	R	0.7	0.4	0:00.07	top
431	root	15	-5	0	0	0	S	0.3	0.0	0:01.05	scsi_eh_2
1950	root	20	0	3388	856	728	S	0.3	0.3	0:01.08	hald-addon-stor
2319	root	39	19	92452	65m	5088	R	0.3	26.4	0:01.09	yum-updatesd-he
[root@alexandre ~]# renice +19 -p 2571											:00.75 init
2571: prioridade antiga = 0; prioridade nova = 19											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2355	root	20	0	120m	73m	13m	R	20.6	29.5	4:19.32	tomboy
2139	root	20	0	36704	8396	4292	S	2.8	3.3	1:00.13	X
426	root	15	-5	0	0	0	S	0.3	0.0	0:01.67	ata/0
431	root	15	-5	0	0	0	S	0.3	0.0	0:03.04	scsi_eh_2
1950	root	20	0	3388	792	712	S	0.3	0.3	0:03.18	hald-addon-stor
2369	root	20	0	85108	17m	9412	S	0.3	6.8	0:02.13	gnome-terminal
2571	root	39	19	2272	1004	780	R	0.3	0.4	0:00.10	top
1	root	20	0	2112	536	512	S	0.0	0.2	0:00.76	init

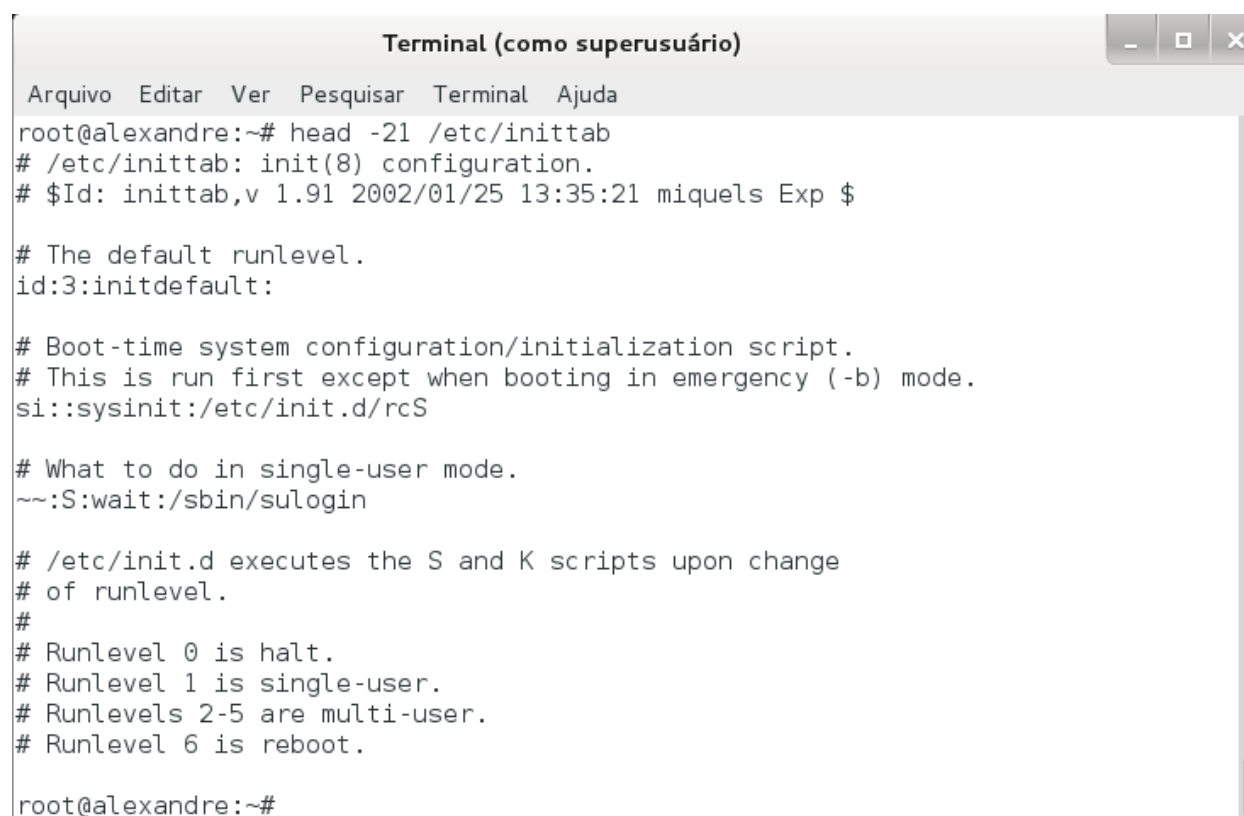
4. Processo de Inicialização

O termo boot (inicializar) vem de *carregador de bootstrap*, em contrapartida, vem da velha declaração “levantar-se por seu próprio esforço”. O significado desta expressão é que você tem que realizar tudo por conta própria, sem qualquer ajuda externa. Este é o termo adequado para um sistema começar do nada e terminar chamando a execução de um sistema operacional completo. Quando o processo de inicialização começa, não há nada na RAM – nenhum programa para carregar o sistema. O carregador que inicia o processo reside na memória não volátil. Em sistemas de PC, isto significa que o carregador faz parte da ROM BIOS.

Inicializar um PC Linux é um processo de várias etapas. Envolve funções de PC básicas, bem como processos de Linux. Este processo complexo começa na ROM BIOS do PC; começa com o programa ROM BIOS que carrega o setor de inicialização a partir do dispositivo de inicialização. O setor de inicialização contém ou carrega um carregador de inicialização do Linux que então carrega o kernel. Finalmente, o kernel começa o processo **init**, que carrega todos os serviços do Linux.

4.1 O processo init

O processo **init** é o processo número um, é a mãe de todos os processos. Depois que o kernel inicializa todos os dispositivos, o programa **init** executa e inicia todo o software. O programa **init** é configurado pelo arquivo **/etc/inittab**. A seguir tem-se o exemplo do arquivo que vem no Debian 7.6:



```

Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# head -21 /etc/inittab
# /etc/inittab: init(8) configuration.
# $Id: inittab,v 1.91 2002/01/25 13:35:21 miquels Exp $

# The default runlevel.
id:3:initdefault:

# Boot-time system configuration/initialization script.
# This is run first except when booting in emergency (-b) mode.
si::sysinit:/etc/init.d/rcS

# What to do in single-user mode.
~~:S:wait:/sbin/sulogin

# /etc/init.d executes the S and K scripts upon change
# of runlevel.
#
# Runlevel 0 is halt.
# Runlevel 1 is single-user.
# Runlevels 2-5 are multi-user.
# Runlevel 6 is reboot.

root@alexandre:~#

```

Como pode-se observar na figura acima, o Linux define vários níveis de execução que executam uma série completa de possíveis estados do sistema de não-executando (parado) até executar múltiplos processos para múltiplos usuários. As linhas comentadas no começo do arquivo **inittab** descrevem seus níveis de execução:

- Nível 0: faz o init fechar todos os processos em execução e cessar o sistema.
- Nível 1: é usado para colocar o sistema em modo monousuário. O modo monousuário é usado pelo administrador do sistema para executar a manutenção que não pode ser feita quando os usuários estão registrados.

- Nível 2 ao nível 5: é um modo multiusuário e inicializa o sistema com a interface gráfica. Este nível de execução é extensamente usado como uma alternativa para sistemas configurados para lançar um ambiente desktop X na inicialização. No nível 2 a interface gráfica (gdm) não inicializa.
- Nível 6: faz o `init` fechar todos os processos em execução e reiniciar o sistema.

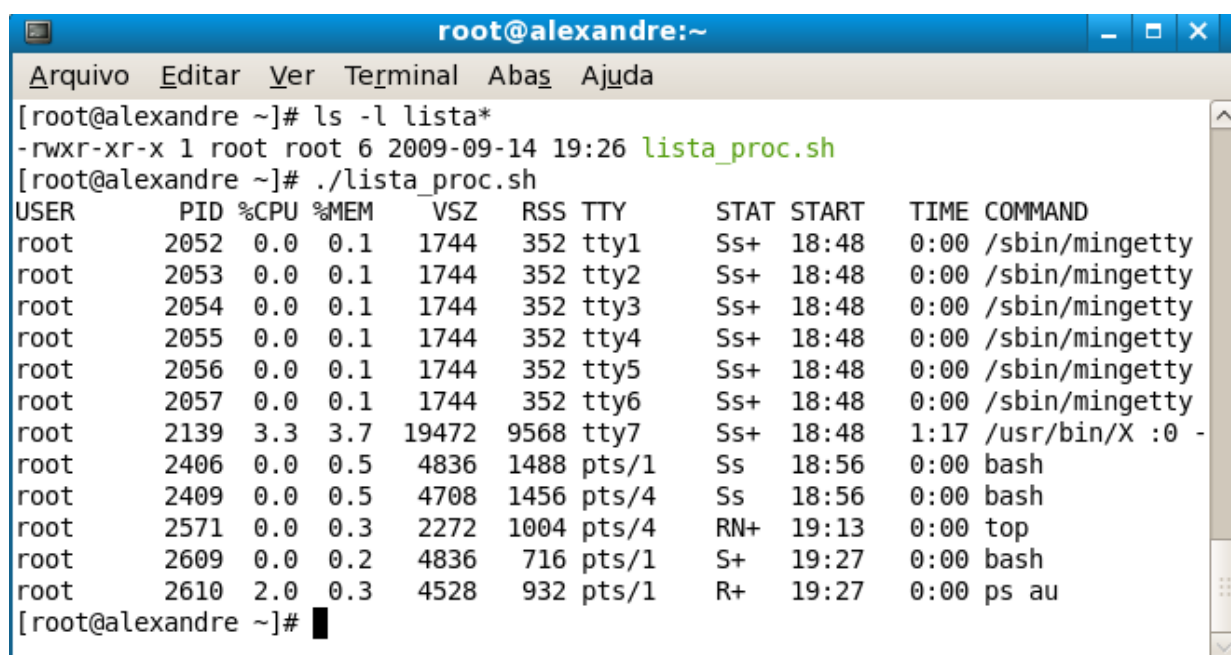
Cada linha no arquivo **inittab** manipula alguma tarefa importante. Porém, o verdadeiro coração do arquivo **inittab** consiste das sete linhas que seguem o comentário “System initialization” (inicialização do sistema) próximo ao começo do arquivo **inittab**. Elas são as linhas que invocam os scripts de inicialização. O primeiro destes é a entrada **si**:

```
# The default runlevel.
id:3:initdefault::
```

Esta linha inicia todos os processos e serviços necessários para fornecer o suporte multiusuário completo definido pelo nível de execução 3.

4.2 Scripts de inicialização

Qualquer coisa que pode ser executada de um prompt shell pode ser armazenada em um arquivo e ser executada como um script shell. Administradores de sistemas usam esta capacidade para automatizar todos os tipos de processos; o Linux usa para automatizar a inicialização de serviços de sistema. A seguir tem-se um exemplo de script:



```
root@alexandre:~
Arquivo Editar Ver Terminal Abas Ajuda
[root@alexandre ~]# ls -l lista*
-rwxr-xr-x 1 root root 6 2009-09-14 19:26 lista_proc.sh
[root@alexandre ~]# ./lista_proc.sh
USER      PID  %CPU  %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      2052  0.0   0.1   1744    352 tty1      Ss+  18:48   0:00 /sbin/mingetty
root      2053  0.0   0.1   1744    352 tty2      Ss+  18:48   0:00 /sbin/mingetty
root      2054  0.0   0.1   1744    352 tty3      Ss+  18:48   0:00 /sbin/mingetty
root      2055  0.0   0.1   1744    352 tty4      Ss+  18:48   0:00 /sbin/mingetty
root      2056  0.0   0.1   1744    352 tty5      Ss+  18:48   0:00 /sbin/mingetty
root      2057  0.0   0.1   1744    352 tty6      Ss+  18:48   0:00 /sbin/mingetty
root      2139  3.3   3.7  19472   9568 tty7      Ss+  18:48   1:17 /usr/bin/X :0 -
root      2406  0.0   0.5   4836   1488 pts/1     Ss   18:56   0:00 bash
root      2409  0.0   0.5   4708   1456 pts/4     Ss   18:56   0:00 bash
root      2571  0.0   0.3   2272   1004 pts/4     RN+  19:13   0:00 top
root      2609  0.0   0.2   4836    716 pts/1     S+   19:27   0:00 bash
root      2610  2.0   0.3   4528    932 pts/1     R+   19:27   0:00 ps au
[root@alexandre ~]#
```

Geralmente é utilizado dois tipos principais de script: o script de inicialização de sistema e o script de inicialização de nível de execução.

O primeiro script executa muitas tarefas de inicialização de sistemas essenciais, como preparar a rede e o sistema de arquivos para uso.

O script de inicialização executa muitas outras tarefas pequenas, mas importantes, como acertar o relógio do sistema, aplicar qualquer mapa de teclado e iniciar o suporte USB e PnP.

Depois que o script de inicialização de sistema é executado, o **init** executa um script para o nível de execução específico. O script executa todos os scripts que são apropriados ao nível de execução. Faz isto executando os scripts que estão armazenados no diretório `/etc/rcn.d` onde **n** é o nível de execução especificado. Por exemplo, se ao script `rc` for passado um 3, executa os scripts encontrados no diretório `/etc/rc3.d`. Abaixo temos a listagem do diretório, onde mostra que existem muitos scripts:

```

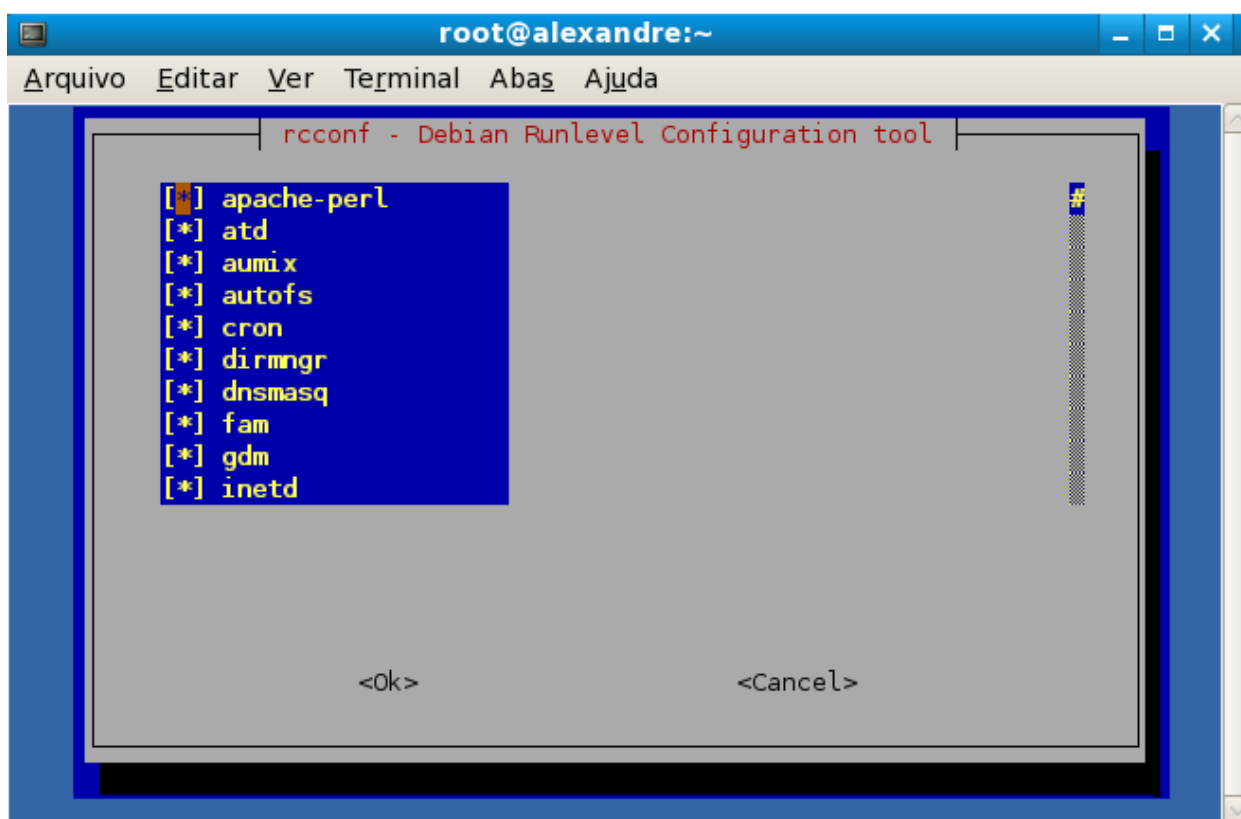
root@alexandre:/etc/rc3.d
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
lrwxrwxrwx 1 root root 11 2008-08-19 12:07 S99local -> ../rc.local
[root@alexandre rc3.d]# ls
K01smartd      K73ypbind      S15mdmonitor
K01smolt       K74lm_sensors  S18rpcidmapd
K02NetworkManager K74nscd       S19rpcgssd
K02NetworkManagerDispatcher K74ntpd      S25netfs
K36postgresql S10network     S98haldaemon
K50netconsole  S11auditd     S99anacron
K50vsftpd      S12restorecond S99firstboot
K69rpcsvcgssd  S13irqbalance S99local

```

Os scripts que se encontram nos diretórios /etc/rc?.d estão no seguinte formato: SNNnome e KNNnome. Onde NN é o número sequencial de inicialização, ou seja, define a ordem de execução. Os scripts que começam com S serão iniciados automaticamente e os que começam com K precisam ser iniciados manualmente, ou seja, não são iniciados automaticamente. Vários destes scripts são claramente de interesse de administradores de servidores de rede:

- O script httpd inicia o servidor web;
- O script named inicia o servidor de nome DNS;
- Sendmail inicia o servidor de e-mail.

Pode-se definir quais serviços podem ser iniciados quando o Sistema Operacional é iniciado, para isso existem diversas formas, por exemplo através dos softwares: **rcconf**, **sysvconfig** e o **sysv-rc-conf**. A seguir a tela do **rcconf**.



Os softwares **sysvconfig** e o **sysv-rc-conf** são completos por definirem por nível de inicialização, a seguir uma figura que mostra tais softwares.

SysV Runlevel Config - : stop service =/+ : start service h: help q: quit								
service	1	2	3	4	5	0	6	S
atd	[]	[X]	[X]	[X]	[X]	[]	[]	[]
bootlogd	[]	[]	[]	[]	[]	[]	[]	[X]
cron	[]	[X]	[X]	[X]	[X]	[]	[]	[]
dbus-1	[]	[X]	[X]	[X]	[X]	[]	[]	[]
dirmngr	[]	[X]	[X]	[X]	[X]	[]	[]	[]
discover	[]	[]	[]	[]	[]	[]	[]	[X]
dns-clean	[]	[]	[]	[]	[]	[]	[]	[X]
exim4	[]	[X]	[X]	[X]	[X]	[]	[]	[]
fam	[]	[X]	[X]	[X]	[X]	[]	[]	[]
gdm	[]	[X]	[X]	[X]	[X]	[]	[]	[]
halt	[]	[]	[]	[]	[]	[X]	[]	[]
hotplug	[]	[]	[]	[]	[]	[]	[]	[X]
hotplug-n\$	[]	[]	[]	[]	[]	[]	[]	[X]

Use the arrow keys or mouse to move around. ^n: next pg ^p: prev pg
 space: toggle service on / off

"Debian Sysv Config Utility"

Edit links for a Service

Use the up and down arrows to move between fields and the left and right arrows to move within a field. You must delete characters with the backspace before entering new ones. Links must consist of the letter K or S followed by two digits. To delete a link just leave the field blank. When you are finished, use TAB to select <OK> and ENTER to move on to the next item.

	0	1	2	3	4	5	6
acct	K20	K20	S20	S20	S20	S20	K20
alsa-utils	K50	-	S50	S50	S50	S50	K50
anacron	K11	K11	S89	S89	S89	S89	K11
autofs	K19	K19	K19	K19	K19	K19	K19
avahi-daemon	K16	K16	S24	S24	S24	S24	K16
bootchart	-	S99	S99	S99	S99	S99	-
bootlogd	-	-	-	-	-	-	-
bootmisc.sh	-	-	-	-	-	-	-
checkfs.sh	-	-	-	-	-	-	-
checkroot.sh	-	-	-	-	-	-	-
console-screen.kbd.sh	-	-	-	-	-	-	-
cron	K11	K11	S89	S89	S89	S89	K11

v(+)

< OK > <Cancel>

4.3 Agendamento de scripts com crontab

Podem-se programar tarefas para serem executadas com certa periodicidade ou em um exato dia e/ou hora, por exemplo, um script que verifique se o roteador de uma rede está funcionando e caso tenha problemas o script envie um e-mail para o administrador da rede.

Arquivos crontab que agendam tarefas de manutenção de sistema e outras tarefas são armazenados no arquivo /etc/crontab e em outros arquivos localizados no diretório /etc/cron.d.

Cada linha contém seis ou sete campos e representa um comando, a seguir tem-se a especificação dos campos:

minuto hora dia mês diaSemana [nomeUsuário] comando

Cada um dos campos relacionados a data/hora pode conter:

- Um asterisco que coincide com tudo;
- Um inteiro único, que coincide exatamente;
- Dois inteiros separados por um traço, coincidindo com um intervalo de valores; e
- Uma série de inteiros ou intervalos separados por vírgulas, que coincide com qualquer valor listado.

Exemplo de datas:

```
05 12 * * 0,6 // significa 12h05m de sábado e domingo
00 02 * * * // significa todo dia às 2 horas
00 23 * 30 * // significa todo dia 30 às 23 horas
```

Exemplo de script:

```

root@urano:~
Arquivo Editar Ver Terminal Ajuda
[root@urano ~]# cat cria_dir.sh
data=`date +%Y%m%d`
mkdir /tmp/backup$data
[root@urano ~]#

root@urano:~
Arquivo Editar Ver Terminal Ajuda
[root@urano ~]# cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
HOME=/

# For details see man 4 crontabs

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * command to be executed
57 14 * * * root /root/cria_dir.sh

[root@urano ~]#

```

O resultado pode ser visto a seguir:

```

root@urano:~
Arquivo Editar Ver Terminal Ajuda
[root@urano ~]# ls -l /tmp
total 4
drwxr-xr-x  2 root      root      4096 Mar  5 14:57 backup20130305
srwxrwxr-x. 1 alexandre alexandre  0 Nov 29 2011 gnome-system-monitor.alexandre.2643626631
[root@urano ~]#

```

É possível também agendar tarefas com o comando **crontab -e**, desta forma não é necessário incluir o nome do usuário (sexto campo) e cria-se um arquivo com o nome do usuário em **/var/spool/cron/crontabs**.

4.4 Outros scripts

É possível automatizar um comando (ou conjunto de comandos) para executar uma tarefa (ou várias tarefas) com o uso de um script. A seguir têm-se dois exemplos de scripts:

```

root@urano:~
Arquivo Editar Ver Terminal Ajuda
[root@urano ~]# cat teste.sh
ls -l $1 > $2
[root@urano ~]#
[root@urano ~]# ./teste.sh /etc a.txt
[root@urano ~]#
[root@urano ~]# ls -lh a.txt
-rw-r--r-- 1 root root 19K Mar  5 15:42 a.txt
[root@urano ~]#

```

```

root@urano:~
Arquivo Editar Ver Terminal Ajuda
[root@urano ~]# cat teste2.sh
for ((i=1; i <= $1; i++)); do
    if ping -c 2 192.168.137.$i >> result.txt ; then
        echo "IP 192.168.137.$i comunicou" >> IP.txt
    else
        echo "IP 192.168.137.$i nao comunicou" >> IP.txt
    fi
done
[root@urano ~]# ./teste2.sh 5
[root@urano ~]# cat IP.txt
IP 192.168.137.1 comunicou
IP 192.168.137.2 comunicou
IP 192.168.137.3 nao comunicou
IP 192.168.137.4 nao comunicou
IP 192.168.137.5 nao comunicou
[root@urano ~]#

```


5. Interface de rede

Vamos tratar neste capítulo como configurar um sistema Linux como um servidor de rede, instalando e configurando sua interface de rede. Nada é mais básico para a configuração de rede do que a interface que o sistema usa para se conectar a rede.

Na maioria dos servidores Linux, a interface de rede é uma placa Ethernet. Uma interface Ethernet do Linux é composta de uma placa adaptadora de hardware e uma driver de software.

As interfaces de rede estão localizadas no diretório `/dev` e a maioria são criadas dinamicamente pelo SO quando instalado ou pelos softwares quando são requisitadas. Este é o caso das interfaces **ppp** e **plip** que são criadas dinamicamente pelos softwares. Abaixo a identificação de algumas interfaces de rede no Linux: (? - significa um número que identifica as interfaces seqüencialmente, iniciando em 0):

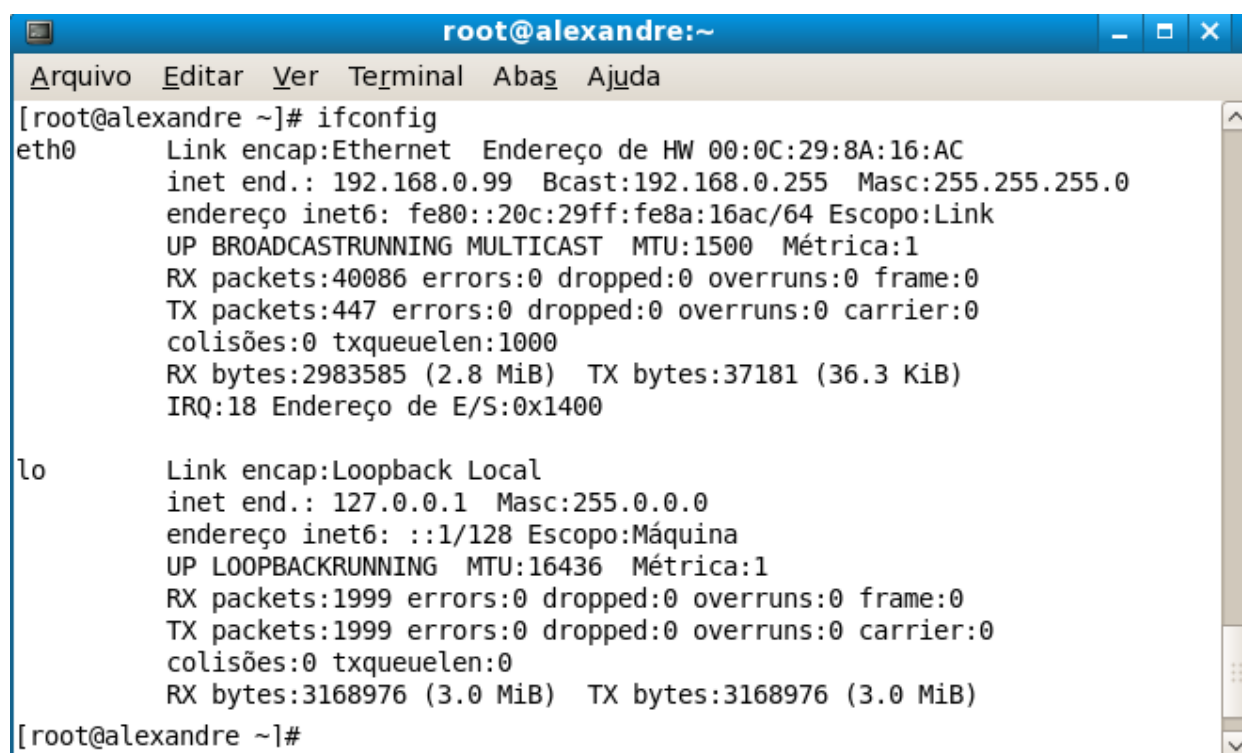
- eth?: Placa de rede Ethernet.
- wlan?: Placa de rede Ethernet sem fio.
- lo: Interface local do computador.
- ppp?: Interface de rede PPP (Placa de Modem).
- slip?: Interface de rede serial
- plip?: Interface de porta paralela

A interface loopback é um tipo especial de interface que permite fazer conexões com você mesmo. Todos os computadores que usam o protocolo TCP/IP utilizam esta interface e existem várias razões para fazer isto, por exemplo, você pode testar vários programas de rede sem interferir com ninguém em sua rede. Por convenção, o endereço IP 127.0.0.1 foi escolhido especificamente para a interface loopback, assim se abrir uma conexão telnet para 127.0.0.1, abrirá uma conexão para o próprio computador local.

5.1 O comando ifconfig

Para instalar um computador usando o Linux em uma rede TCP/IP é necessário ter algumas informações antes, as informações principais são: Endereço IP do seu computador, Máscara da Rede IP, Endereço IP do Roteador, Endereço do Servidor de Nomes (DNS). As duas últimas informações são necessárias para o computador ter acesso à Internet.

O comando **ifconfig** mostra as interfaces de rede ativas, bem como, configura, ativa e desativa uma interface de rede específica. Abaixo um exemplo com o comando **ifconfig**:



```

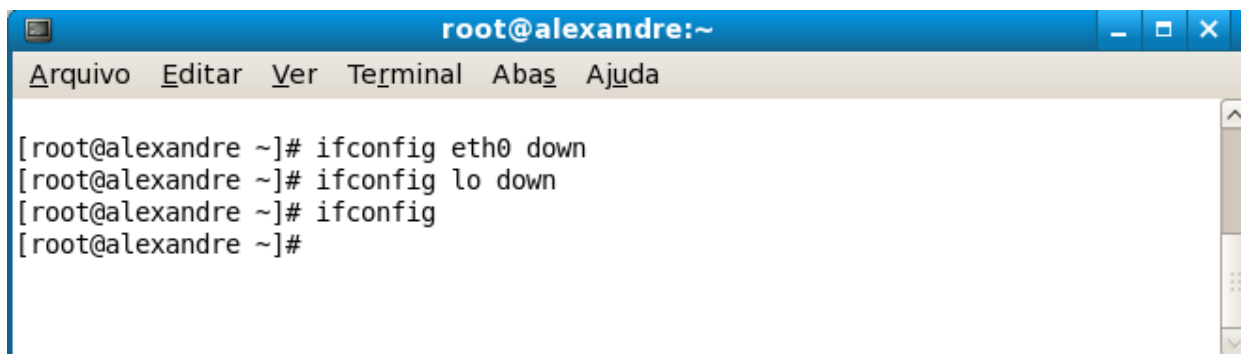
root@alexandre:~
Arquivo  Editar  Ver  Terminal  Abas  Ajuda
[root@alexandre ~]# ifconfig
eth0      Link encap:Ethernet  Endereço de HW 00:0C:29:8A:16:AC
          inet end.: 192.168.0.99  Bcast:192.168.0.255  Masc:255.255.255.0
          endereço inet6: fe80::20c:29ff:fe8a:16ac/64  Escopo:Link
          UP BROADCASTRUNNING MULTICAST  MTU:1500  Métrica:1
          RX packets:40086 errors:0 dropped:0 overruns:0 frame:0
          TX packets:447 errors:0 dropped:0 overruns:0 carrier:0
          colisões:0 txqueuelen:1000
          RX bytes:2983585 (2.8 MiB)  TX bytes:37181 (36.3 KiB)
          IRQ:18  Endereço de E/S:0x1400

lo        Link encap:Loopback Local
          inet end.: 127.0.0.1  Masc:255.0.0.0
          endereço inet6: ::1/128  Escopo:Máquina
          UP LOOPBACKRUNNING  MTU:16436  Métrica:1
          RX packets:1999 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1999 errors:0 dropped:0 overruns:0 carrier:0
          colisões:0 txqueuelen:0
          RX bytes:3168976 (3.0 MiB)  TX bytes:3168976 (3.0 MiB)

[root@alexandre ~]#

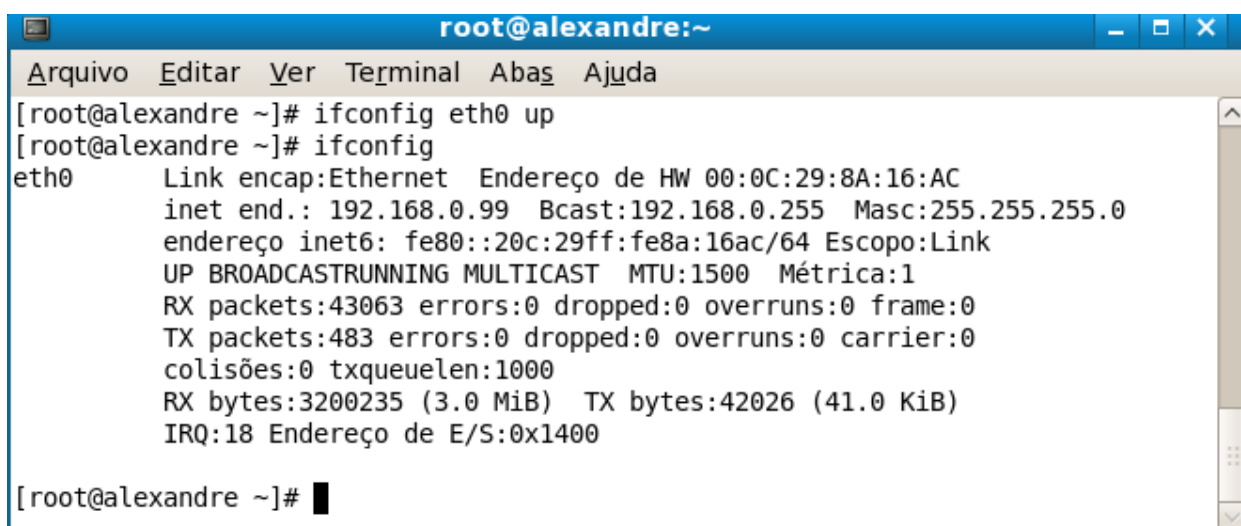
```

Abaixo temos um exemplo desativando as interfaces eth0 e lo, com o comando **ifconfig eth0 down**:



```
root@alexandre:~  
Arquivo Editar Ver Terminal Abas Ajuda  
[root@alexandre ~]# ifconfig eth0 down  
[root@alexandre ~]# ifconfig lo down  
[root@alexandre ~]# ifconfig  
[root@alexandre ~]#
```

Agora temos um exemplo ativando a interface eth0, com o comando **ifconfig eth0 up**:



```
root@alexandre:~  
Arquivo Editar Ver Terminal Abas Ajuda  
[root@alexandre ~]# ifconfig eth0 up  
[root@alexandre ~]# ifconfig  
eth0      Link encap:Ethernet  Endereço de HW 00:0C:29:8A:16:AC  
          inet end.: 192.168.0.99  Bcast:192.168.0.255  Masc:255.255.255.0  
          endereço inet6: fe80::20c:29ff:fe8a:16ac/64  Escopo:Link  
          UP BROADCASTRUNNING MULTICAST  MTU:1500  Métrica:1  
          RX packets:43063 errors:0 dropped:0 overruns:0 frame:0  
          TX packets:483 errors:0 dropped:0 overruns:0 carrier:0  
          colisões:0 txqueuelen:1000  
          RX bytes:3200235 (3.0 MiB)  TX bytes:42026 (41.0 KiB)  
          IRQ:18  Endereço de E/S:0x1400  
[root@alexandre ~]#
```

Os comandos de ativação e desativação podem ser substituídos respectivamente pelos comandos **ifup [interface]** e **ifdown [interface]**.

Para configurar uma interface de rede utilizando o comando **ifconfig** e supondo-se que o endereço IP da interface **eth0** é 192.168.0.10 com a máscara de rede 255.255.255.0, o comando seria:

```
ifconfig eth0 192.168.0.10 netmask 255.255.255.0
```

Uma observação importante é que ao configurar ou mudar o endereço IP de uma interface de rede pelo **ifconfig**, muda-se apenas naquele momento, ou seja, fica somente na memória inclusive perde-se as rotas da rede. Para fixar a mudança, é necessário alterar o arquivo de configuração específico da interface e reiniciar o serviço de rede.

Abaixo temos a localização dos principais arquivos de configuração de rede utilizada pelos serviços de rede:

- /etc/resolv.conf**: arquivo de configuração dos servidores DNS primário e secundário;
- /etc/hostname**: configuração do nome da máquina;
- /etc/network/interfaces**: arquivo para configuração da interface de rede.


```
Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~# cat /etc/resolv.conf
nameserver 192.168.1.1
nameserver 200.204.0.10
nameserver 192.168.1.2
root@alexandre:~#
root@alexandre:~# cat /etc/hostname
alexandre
root@alexandre:~#
root@alexandre:~#
root@alexandre:~#
```

```
Terminal (como superusuário)
Arquivo Editar Ver Pesquisar Terminal Ajuda
root@alexandre:~#
root@alexandre:~# tail -20 /etc/network/interfaces

# The loopback network interface
auto lo

iface lo inet loopback

auto eth0
iface eth0 inet static
address 192.168.137.10
netmask 255.255.255.0
gateway 192.168.137.1
network 192.168.137.0
broadcast 192.168.137.255
#dns-nameservers 192.168.137.1

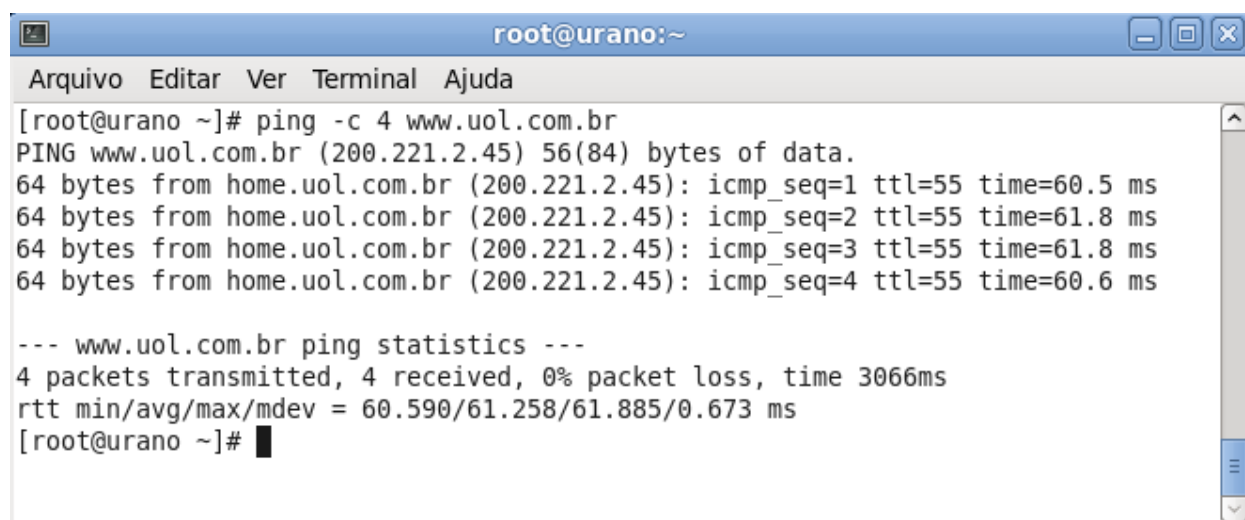
auto eth1
iface eth1 inet dhcp

auto eth2
iface eth2 inet dhcp
root@alexandre:~#
```

Podem-se usar os arquivos acima para alterar as configurações de rede, depois de alterado deve-se reiniciar o serviço de rede para aplicar as alterações efetuadas. Para reiniciar o serviço, executamos o comando **service network restart**. Pode-se também ativar e desativar a interface, com os comandos **ifdown** e **ifup**.

5.2 O comando ping

O comando **ping** (**P**acket **I**Nternet **G**rouper) é usado para testar a conexão de rede. O **ping** é simples, pois não depende de qualquer configuração de rede. Um teste bem sucedido lhe informa que o sistema remoto está alcançável. Isto significa que a configurações das interfaces do sistema local e remoto estão corretas.

A terminal window titled 'root@urano:~' with a menu bar containing 'Arquivo', 'Editar', 'Ver', 'Terminal', and 'Ajuda'. The terminal displays the output of a ping command. The command executed is '[root@urano ~]# ping -c 4 www.uol.com.br'. The output shows four successful ping requests from 'home.uol.com.br' (200.221.2.45) with varying times and TTL values. It also displays summary statistics: 4 packets transmitted, 4 received, 0% packet loss, and a total time of 3066ms. The round-trip times (rtt) are listed as min/avg/max/mdev = 60.590/61.258/61.885/0.673 ms. The prompt '[root@urano ~]#' is followed by a cursor.

```
root@urano:~  
Arquivo  Editar  Ver  Terminal  Ajuda  
[root@urano ~]# ping -c 4 www.uol.com.br  
PING www.uol.com.br (200.221.2.45) 56(84) bytes of data.  
64 bytes from home.uol.com.br (200.221.2.45): icmp_seq=1 ttl=55 time=60.5 ms  
64 bytes from home.uol.com.br (200.221.2.45): icmp_seq=2 ttl=55 time=61.8 ms  
64 bytes from home.uol.com.br (200.221.2.45): icmp_seq=3 ttl=55 time=61.8 ms  
64 bytes from home.uol.com.br (200.221.2.45): icmp_seq=4 ttl=55 time=60.6 ms  
  
--- www.uol.com.br ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3066ms  
rtt min/avg/max/mdev = 60.590/61.258/61.885/0.673 ms  
[root@urano ~]#
```

6. Bibliografia

Linux: Servidores de Rede / Craig Hunt. 1ª Edição - Rio de Janeiro: Ciência Moderna, 2004.

Linux: Redes e Servidores – Guia Prático / Carlos E. Morimoto, 2ª Edição – Porto Alegre: GDH Press e Sul Editores, 2006.

Squid: Configurando o Proxy para Linux / Antonio Marcelo. 5ª Edição – Rio de Janeiro: Brasport, 2006.

Apache: Configurando o servidor WEB para Linux / Antonio Marcelo. 3ª Edição – Rio de Janeiro: Brasport, 2006.